INVOISUITE · DEPLOYMENT GUIDE

Hosting InvoiSuite on your own server

From an empty server to a live, secure backend and a release build of the Android app.

What needs hosting

Only the **backend** (Spring Boot + an Oracle database) needs a server. The Android app is a client that you distribute as an APK or through the Play Store; it simply talks to the backend over HTTPS. The simplest setup is **one small VPS** running the project's existing `docker-compose.yml`.

Part	Where it runs
Backend API + Oracle database	Your VPS, in Docker (`invoisuite-server/docker-compose.yml`)
HTTPS certificate + public entry point	Caddy on the same VPS
Android app	Users' phones, installed from an APK or the Play Store

Steps 1–6 get the backend live. Steps 7–8 cover the Android app. Each step works without the later ones. Prices below are rough; check current provider pricing.

1 Get a server and a domain

- **Server:** rent a small Linux VPS (Hetzner, DigitalOcean or similar). Choose **Ubuntu 24.04** with **4 GB RAM** and 40 GB+ disk, because the Oracle container alone wants about 2 GB. Expect roughly €5–10 per month. The Oracle image has only been run on ARM (an Apple Silicon Mac) during development; x86 is its main target and should work as well.
- **Domain:** buy one (about €10 a year) and add an **A record**, for example `api.yourdomain.com`, pointing to the server's IP address. You need a real domain because the release app only allows HTTPS.
- **Login:** when creating the VPS, add your SSH public key so you log in with a key instead of a password.

2 Prepare the server

Connect to the server:

```
ssh root@YOUR_SERVER_IP
```

Update the system:

```
apt update && apt upgrade -y
```

Install Docker:

```
curl -fsSL https://get.docker.com | sh
```

Set up the firewall so only SSH and web traffic can get in:

```
ufw allow OpenSSH && ufw allow 80 && ufw allow 443 && ufw enable
```

Install **Caddy**, a web server that obtains and renews HTTPS certificates automatically. Follow the Debian/Ubuntu section at `caddyserver.com/docs/install`.

3 Get the code onto the server

The repositories are on GitHub, so the server needs read access. Create a key on the server:

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""
```

Show the public key:

```
cat ~/.ssh/id_ed25519.pub
```

Copy that output into GitHub: your server repository → **Settings** → **Deploy keys** → **Add deploy key** (read-only is enough). Then clone:

```
git clone git@github.com:MarinKatava/invosuite-server.git
```

4 Configure secrets and lock down ports

Inside the `invosuite-server` folder, create a file named `.env`. Docker Compose reads it automatically, and it must never be committed to git.

```
ORACLE_PASSWORD=<long random value>
DB_USERNAME=invoisuite
DB_PASSWORD=<long random value>
JWT_SECRET=<long random value>
ENCRYPTION_KEY=<long random value>
GOOGLE_CLIENT_ID=<only if you use Google sign-in>
```

Generate each random value with:

```
openssl rand -base64 48
```

> **Two things that matter here**
>
> - **Set the database passwords before the first start.**
>   Oracle only uses them when it first creates its data volume. Changing them later does not change the database.
> - **Save `ENCRYPTION_KEY` in a password manager.**
>   It encrypts the stored Odoo API keys. If you lose it, those credentials can no longer be decrypted and users would have to enter them again.

Leave `ALLOW_PRIVATE_HOSTS` and `ALLOW_INSECURE_HTTP` unset (off). They exist only for local development.

Stop publishing the database and backend to the internet

Edit `docker-compose.yml`. Docker publishes ports straight past the firewall, so this change is what actually keeps them private.

Service	Change
`backend`	Change `"8080:8080"` to `"127.0.0.1:8080:8080"` (only the local Caddy can reach it)
`db`	Delete the `"1521:1521"` line (Oracle is reachable only by the backend, inside Docker)

5 Start it

```
docker compose up -d --build
```

The first start takes a few minutes while Oracle initialises. Watch the progress with:

```
docker compose logs -f
```

It is ready when the backend logs *Started InvoisuiteServerApplication*. Press `Ctrl+C` to stop watching the logs (the containers keep running).

6 Add HTTPS

Edit `/etc/caddy/Caddyfile` so it contains only this (use your own domain):

```
api.yourdomain.com {
    reverse_proxy localhost:8080
}
```

Apply it:

```
systemctl reload caddy
```

Test from your own computer:

```
curl https://api.yourdomain.com/api/meta/currencies
```

If you get a list of currencies back, the backend is live. If the certificate fails, check that the DNS record points to the server and that ports 80 and 443 are open.

> **Backups**
> Oracle's data lives in a Docker volume, so it will not survive losing the server. The easiest protection is your provider's automatic server snapshots (usually about 20% extra on the price): turn them on. A proper nightly Oracle export (Data Pump) can be scripted later for point-in-time safety.

7 Point the Android app at your server

In `invoisuite-android/app/build.gradle`, change the **release** build's `API_BASE_URL` to your domain. Keep the trailing slash:

```
release {
    buildConfigField "String", "API_BASE_URL", "\"https://api.yourdomain.com/\""
    ...
}
```

If you use Google sign-in, also put your real web client ID in `google_web_client_id` in the same file, and register your release key's SHA-1 fingerprint in Google Cloud. Email and password sign-in works without any of this.

8 Build and share the app

Create a signing key (once)

```
keytool -genkey -v -keystore invoisuite-release.jks -keyalg RSA -keysize 2048 -validity 1
0000 -alias invoisuite
```

> **Back up the key file and its password.**
> If you lose them you can no longer ship updates to the same app. Release signing is not yet configured in `build.gradle`; that is a small addition when you reach this step.

Build

For an APK you can send directly:

```
./gradlew assembleRelease
```

For the Play Store format (an App Bundle):

```
./gradlew bundleRelease
```

Distribute

- **Privately:** send the APK; people open it and allow installs from unknown sources.
- **Play Store:** needs a developer account (a one-time fee of about $25). New personal accounts may also have to run a closed test with testers before going public, so check the current rules.

Day-to-day operations

Task	Command (run in the `invosuite-server` folder)
Update to the latest code (database stays intact)	`git pull && docker compose up -d --build`
See what is running	`docker compose ps`
Read the backend's logs	`docker compose logs -f backend`
Restart the backend	`docker compose restart backend`
Stop everything (data is kept)	`docker compose stop`

> **Never run `docker compose down -v` on the server.**
> The `-v` flag deletes the database volume and all your data.

Final checklist

- ☐ Domain's A record points at the server
- ☐ Firewall allows only SSH, 80 and 443
- ☐ `.env` contains long random secrets; `ENCRYPTION_KEY` saved in a password manager
- ☐ Ports 8080 (bound to 127.0.0.1) and 1521 (removed) are not public
- ☐ `https://api.yourdomain.com/api/meta/currencies` responds
- ☐ Server snapshots or backups enabled
- ☐ Release `API_BASE_URL` updated; signing key backed up
